

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #6 - Semana 7 – Sivarama - Capitulo 9
Un primer vistazo al lenguaje ensamblador

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 09/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 187–210.

Abstract—*This paper provides an overview of Assembly language, focusing on executable instructions, assembler directives, and macros. It covers data assignment with directives like DB, DW, and DD, and initialization with TIMES. Operand addressing modes, including register, immediate, direct, and indirect, are explained along with the LEA instruction. Key instructions such as MOV, arithmetic operations, and control structures (JMP, LOOP) are detailed, along with logical operations and their role in decision-making.*

I. Instrucciones y Directivas

En lenguaje ensamblador, se utilizan tres tipos principales de declaraciones: instrucciones, directivas de ensamblador y macros. Las instrucciones realizan operaciones, las directivas configuran el ensamblador y las macros simplifican la escritura del código.

Instrucciones de Intercambio y Traducción

a) Instrucción XCHG

- **Propósito:** Intercambia dos valores de datos (8, 16 o 32 bits).
- **Ejemplo:**

```
xchg EAX, EDX
```

```
xchg [response], CL
```

Esto intercambia el valor en EAX con el valor en EDX, o el contenido de la dirección [response] con CL.

Uso en ordenamiento:

xchg [array1], [array2] ; Intercambiar valores para ordenar

b) Instrucción XLAT

- **Propósito:** Traduce un valor de byte usando una tabla de traducción.
- **Ejemplo:**

xlatb

EBX debe contener la dirección base de la tabla y AL el índice. La instrucción reemplaza AL con el byte leído desde la tabla.

Instrucciones de Desplazamiento y Rotación

a) Instrucción SHL (Shift Left)

- **Propósito:** Desplaza los bits hacia la izquierda. El bit más a la izquierda se mueve al flag de acarreo (CF).
- **Ejemplo:**

shl EAX, 1

Desplaza los bits en EAX una posición a la izquierda.

b) Instrucción SHR (Shift Right)

- **Propósito:** Desplaza los bits hacia la derecha. El bit más a la derecha se mueve al flag de acarreo (CF).
- **Ejemplo:**

shr EAX, 1

Desplaza los bits en EAX una posición a la derecha.

c) Instrucciones de Rotación

- **Propósito:** Mantiene los bits que se desplazan fuera del registro al rotarlos hacia el otro extremo.
- **Ejemplos:**
 - **ROL (Rotate Left):**

rol EAX, 1

Rote los bits en EAX hacia la izquierda.

- **ROR (Rotate Right):**

ror EAX, 1

Rote los bits en EAX hacia la derecha.

- **Rotación a través del acarreo:**

rcl EAX, 1

rcr EAX, 1

Definición de Constantes

a) Directiva EQU

- **Propósito:** Define constantes numéricas que no se pueden redefinir.
- **Ejemplo:**

CR EQU 0DH

CR se define como el valor ASCII del carácter de retorno de carro.

b) Directiva %assign

- **Propósito:** Define constantes numéricas que pueden ser redefinidas.
- **Ejemplo:**

%assign MAX_SIZE 100

; Más adelante en el código

%assign MAX_SIZE 200

c) Directiva %define

- **Propósito:** Define constantes tanto numéricas como de cadena, permitiendo redefinición.
- **Ejemplo:**

%define BUFFER_SIZE 256

; Re-definición

%define BUFFER_SIZE 512

Macros

a) Macros sin Parámetros

- **Propósito:** Simplifican el código al sustituir un bloque de texto repetitivo.
- **Ejemplo:**

%macro multEAX_by_16 0

sal EAX, 4

%endmacro

b) Macros con Parámetros

- **Propósito:** Permiten mayor flexibilidad al usar parámetros en las macros.
- **Ejemplo:**

```
%macro mult_by_16 1  
sal %1, 4  
%endmacro
```

; Uso

```
mult_by_16 EAX
```

c) Ejemplo de Macro para Transferencia de Datos

- **Propósito:** Intercambia valores de dos variables de memoria.
- **Ejemplo:**

```
%macro mxchg 2  
xchg EAX, %1  
xchg EAX, %2  
xchg EAX, %1  
%endmacro
```